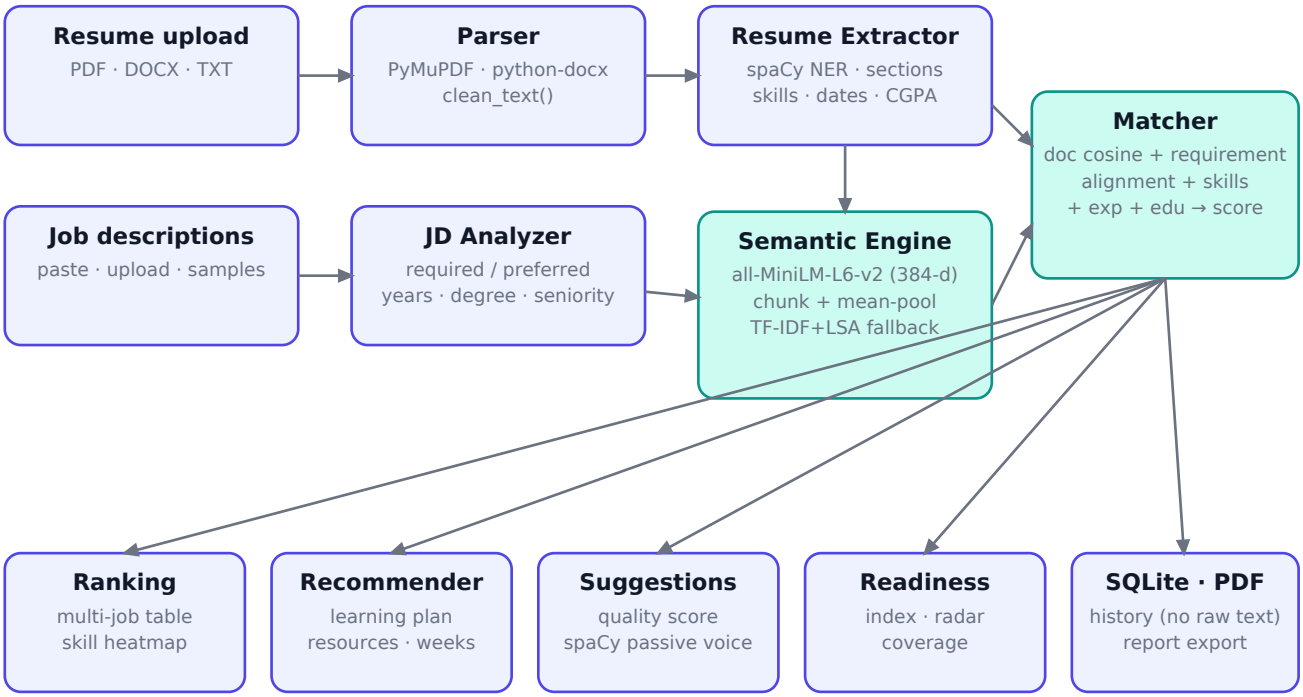
Technical Approach Report

Cynbit Studio – AI Resume & Job Matching Intelligence · 30 October 2026 · evaluation engine: **TF-IDF + LSA (offline fallback)**

1. Objectives

Build a modular, testable system that (1) parses resumes, (2) extracts skills and experience, (3) analyses job descriptions, (4) performs real semantic matching with Sentence Transformers, (5) explains the score, (6) identifies missing skills, (7) recommends learning, (8) compares and ranks jobs, (9) visualises career readiness and (10) suggests resume improvements.

2. Architecture



All AI logic lives in the framework-independent package `src/cynbit`; `app.py` is a thin Streamlit layer. The same `analyze()` function powers the UI, the CLI and the tests, which makes a later FastAPI service trivial.

3. Pipeline

3.1 Parsing

PDFs are read with PyMuPDF using `get_text('text', sort=True)` for natural reading order in multi-column layouts; DOCX paragraphs and table cells are read with python-docx. Text is NFKC-normalised, ligatures (fi, fl) expanded, bullet glyphs unified, whitespace collapsed. Empty, encrypted, corrupted, unsupported or image-only files raise a user-friendly ParsingError.

3.2 Resume information extraction

- **Sections:** header detection against ~60 canonical aliases (Experience, Internships, Projects…).
- **Skills:** taxonomy of 90 canonical skills / ~430 surface forms in 13 categories. Custom regex boundaries `(?<![\w+#])alias(?![\w+#]|\.\w)` match C++, C#, Node.js, CI/CD while

rejecting 'reactive'→React or 'javascript'→Java. Ambiguous names (R, Go) count only in list context.

- **Experience:** date ranges (Jan 2023 – Present, 06/2021 – 08/2022, 2019 – 2021) parsed to fractional years; overlapping intervals merged so concurrent roles are not double-counted.
- **Education:** degree level (Diploma→PhD) with false-positive-safe patterns; CGPA normalised to /10.
- **spaCy (en_core_web_sm):** sentence segmentation, ORG entities, noun-chunk key phrases and dependency-based passive-voice detection (nsubjpass/auxpass).

3.3 Job description analysis

Lines are bucketed into requirements, preferred and responsibilities by headers ("Nice to have", "Requirements"…) and, per sentence, by cue phrases ("is a plus", "familiarity with"). Skills found only in preferred text become *preferred*; all others are *required*. Minimum years, minimum degree and seniority are extracted with targeted patterns.

3.4 Semantic matching with all-MiniLM-L6-v2

all-MiniLM-L6-v2 is a 6-layer, 384-dimensional sentence-embedding model (~22.7M parameters) trained contrastively on over a billion sentence pairs. Because it truncates input at 256 word-pieces, documents are split into ~90-word chunks, embedded, mean-pooled and re-normalised. Two signals are combined:

```
doc_sim = cos( mean_pool(E(resume_chunks)), mean_pool(E(job_chunks)) )
req_cov = mean over job requirements r_i of max_j cos(E(r_i), E(resume_line_j))
semantic = 0.5·cal(doc_sim) + 0.5·cal(req_cov)
```

The requirement-level alignment is what makes the score explainable: for each requirement the UI shows the single resume line that best supports it, and flags weakly-covered requirements. `cal()` linearly maps the model's typical cosine range (≈0.20 unrelated → 0.75 strongly related) to 0–1; the raw cosine is always shown alongside.

Offline fallback. If the model cannot be loaded, a TF-IDF (1–2-grams, sublinear tf) + LSA (TruncatedSVD, 60 components, background corpus built from the skill taxonomy) engine is used. Canonical skill tokens are appended to each text so that aliases such as 'k8s' and 'Kubernetes' coincide. It has its own calibration range (0.05–0.55). The UI warns when the fallback is active.

3.5 Scoring

```
overall = 0.40·semantic + 0.35·skills + 0.15·experience + 0.10·education
skills = Σ w·credit / Σ w ; w = 2 (required), 1 (preferred)
credit = 1 matched · 0.5 related · 0 missing
experience = min(1, years / required_years) (internships add 0.25 yr each if no dated roles)
education = 1 if degree ≥ required; 0.6 one level below; 0.3 otherwise; 0.5 if not detected
```

Related skills receive partial credit through taxonomy relations (TensorFlow ↔ PyTorch, FastAPI → REST APIs) and, with MiniLM, through embedding similarity ≥ 0.62 between skill names in the same category. Verdicts: ≥75 Strong, ≥60 Good, ≥45 Moderate, else Low.

3.6 Recommendations, suggestions and readiness

- **Learning plan:** gaps are aggregated over the 3 best-matching jobs; priority = Σ importance × gap-type × job-fit weight; effort from taxonomy hours (halved for related skills); scheduled into weeks at the student's chosen hours/week with curated free resources and a portfolio project idea.
- **Resume quality:** 15+ checks (structure, contact links, length, quantified-bullet ratio, action verbs, weak phrases, pronouns, buzzwords, passive voice, unevidenced skills, job keywords,

degree); score = 100 − 12/6/3 per high/medium/low issue.

- **Readiness index:** 35% market fit (mean of top-3 matches), 20% skill depth, 20% resume quality, 15% practical experience, 10% profile completeness; plus per-category skill coverage.

4. Data, storage and privacy

SQLite tables `resumes` (SHA-256 fingerprint, filename, derived skills, quality) and `analyses` (scores, verdict, engine, missing skills). Raw resume text is never persisted — a unit test asserts this. Gemini calls (optional) receive text with emails, phones and URLs redacted. Secrets live only in a git-ignored `.env`.

5. Evaluation on the synthetic dataset

Four synthetic resumes (ML student, full-stack developer, data analyst, and a deliberately weak resume) were scored against six synthetic job descriptions using **TF-IDF + LSA (offline fallback)**. Bold marks each resume's top-ranked job.

Resume \ Job	AI/ML Engineer Intern	Backend Java Developer	Data Analyst Intern	Full Stack Developer	MLOps Engineer	NLP Engineer Junior
ananya rao ml student	**82**	44	56	38	53	73
karthik menon fullstack	34	50	40	**89**	49	34
meera iyer data analyst	33	18	**77**	19	18	24
rohan das needs work	30	13	**34**	17	16	15

Top-1 domain accuracy: 3/3 resumes with a known target domain ranked a job from that domain first. **Mean match latency:** 96 ms per resume–job pair on CPU (including requirement-level alignment).

Resume	Readiness	Level	Resume quality	#Suggestions
ananya rao ml student	85	Job-ready	94	1
karthik menon fullstack	81	Job-ready	82	2
meera iyer data analyst	66	Almost ready	82	2
rohan das needs work	22	Early stage	0	14

The weak resume receives the lowest readiness and quality scores and the most suggestions, and every domain resume scores its own domain far above unrelated roles — evidence that scores respond to content rather than being fixed. A synthetic set this small validates behaviour, not accuracy; Section 8 describes the labelled evaluation needed before production.

6. Testing

The pytest suite (44 tests) covers parsing (PDF/DOCX/TXT, corrupted/empty/unsupported files), skill extraction edge cases (C++, Node.js, false positives, R/Go), date-range merging, education detection, JD required/preferred separation, similarity properties, domain-ranking sanity, related-skill credit, recommendations scheduling, suggestions, readiness ordering, SQLite privacy, PDF export, and a headless Streamlit `AppTest` that renders all eight pages in empty and demo states. A MiniLM-specific test is marked `transformer` and skips automatically when the model is not cached. A GitHub Actions workflow runs the suite on every push.

7. Technology choices

Concern	Choice	Reason
Embeddings	Sentence Transformers all-MiniLM-L6-v2	Strong quality/speed trade-off; CPU-friendly; 384-d
Linguistics	spaCy en_core_web_sm	Fast sentence splitting, NER, dependency parse for passive voice
Parsing	PyMuPDF, python-docx	Accurate text order, table support, no external services
Fallback	scikit-learn TF-IDF + TruncatedSVD	Works fully offline, deterministic
UI / charts	Streamlit, Plotly	Rapid, interactive, testable with AppTest
Storage	SQLite (stdlib)	Zero-config; swap for PostgreSQL + pgvector in production
Optional GenAI	Google Gemini (google-genai)	Coaching only; never affects scores

8. Limitations and future work

- Calibration ranges and weights are heuristic → fit on labelled resume–job pairs (logistic regression over components) and report precision@k / NDCG.
- Taxonomy-bound skill extraction → add a spaCy span-categorizer trained on job postings, with embedding nearest-neighbour mapping to canonical skills.
- No OCR for scanned PDFs → add Tesseract/PaddleOCR fallback.
- English only → multilingual MiniLM (paraphrase-multilingual-MiniLM-L12-v2) for Indian-language resumes.
- Single-user local app → FastAPI service, authentication, PostgreSQL + pgvector, background workers.

9. Reproducibility

```
pip install -r requirements-dev.txt
python scripts/download_models.py
pytest
python scripts/generate_reports.py # regenerates both PDFs with live evaluation
streamlit run app.py
```